PS-PL AXI HP 接口：事务计数与复位语义（客户说明）

面向对象：使用该芯片 HP 端口的软硬件工程师 **文档性质**：面向客户的说明稿 **日期**：2026-10-12

速览（先看这一页）

一句话结论：该芯片的 PS 侧不提供"在途事务计数"信号，也不暴露"读通道是否已清空"的状态。要判断接口是否已排空，只能由 PL 侧自己计数、或由 PL master 输出忙闲状态。

五条要点

1. **计数**：PS 侧没有 `S_AXI_HP*_RCOUNT` / `RACOUNT`，AXI 协议也不定义这类信号，只能在 PL 侧自行实现。
2. **复位**：AXI 不要求"复位时把在途事务做完"。任一侧在非空闲时被复位，另一侧会一直等下去（挂死）。
3. **场景**：复位落在哪一层，结局完全不同（下表 L1–L5）。只有"PS 与 PL 一起清"才没有残留。
4. **处置**：凡是"只动 PL"的操作（PL 重配置、PL 逻辑复位），都必须**先排空**；而判空只能靠 PL 自己。
5. **判空**：推荐给 PL master 加"使能输入 + 忙/闲状态输出"（下称**判空握手**），用它决定什么时候可以复位。

五种场景速查

场景	会发生什么	怎么办
L1 PS 系统复位	链路被切断，PL 若不作处理会挂死	由软件复位 PL（见 2.4 L1）
L2 只写 `AXI_HP_RST_CTRL`	只清 HP 侧 NIC 桥，下游残留清不掉	必须在排空之后使用
L3 PL 重配置	PS 侧在途事务无人消费，总线访问被阻塞	重配置前必须先排空
L4 PL 逻辑复位	同 L3，但配置保留、恢复快	同样先排空
L5 整机上下电、PS 与 PL 成对复位	两侧全清，无残留	最安全，L1 的流程同样适用

怎么读本文

- 只想知道"能不能做、该怎么做"：看本页与 2.4（各场景怎么做）。
- 要在 PL 里自己写计数器：看第 1 部分。
- 设备偶发挂死、想定位原因：看 2.3（各场景机理）与 2.5（客户问答）。

术语与缩写

词	含义
PS / PL	处理系统（ARM 侧）/ 可编程逻辑（FPGA 侧）

词	含义
设计1 / 设计2 / 设计3	三种 PL 时钟 / 复位接法：设计1 都取自 PS；设计2 复位取自 PS、时钟取自管脚；设计3 都与 PS 无关。**该分档仅适用于 PL 复位采用 `proc_sys_reset` IP 的设计**，其他接法需具体分析（见 2.2）
HP 端口	高性能 AXI 端口，PL 通过它访问 PS 侧资源（如 DDR）
AFI	客户来函使用的名称，指 PS 侧连接 HP 端口的接口模块。该芯片的 PS 内不含此模块；与"AFI 复位"对应的是 **HP 端口侧的 NIC 桥**
NIC 桥	HP 端口侧的一段桥接逻辑，`AXI_HP_RST_CTRL` 复位的是它
DDRC	DDR 控制器，位于 NIC 桥下游
在途事务（outstanding）	已发出请求、尚未收到最终响应的 AXI 事务
背压	下游不接收数据，导致端口内数据排队占满，进而使总线上其他访问被阻塞
判空握手	PL master 对外给出"是否还有未完成事务 / 总线是否空闲"的状态
master / slave	AXI 事务的发起方（master，本文场景是 PL）/ 应答方（slave，本文场景是 PS 侧）
beat	AXI 通道上一次握手所传输的数据单元（本文计数是事务级，不数 beat）
System ILA	FPGA 内部逻辑分析仪；其 "AXI4 Transaction Tracking" 带事务计数功能
位流（bitstream）	PL 的配置文件；PL 重配置就是重新加载它
`FCLK` / `FCLK_RESETN`	PS 提供给 PL 的时钟 / 复位信号
`FPGA_RST_CTRL`	PL 复位控制寄存器（写它断言 / 释放 PL 复位）
POR / SRST	PS 的两种复位源：上电复位 / 系统复位
`USER_LEVEL_SHIFT`	PS-PL 连通开关。PS 复位后会自动清零，PS→PL 与 PL→PS 的信号都随之被切断并钳位
`AXI_HP_RST_CTRL`	写入该寄存器可复位 HP 端口侧的 NIC 桥
`PROGRAM_B`	触发 PL 重配置（重新加载位流）
L1–L5	本文给五种复位场景编的号

1. 计数：能读到"还剩多少在途事务"吗

这一节回答：`RCOUNT` / `RACOUNT` 是什么、能不能用；要自己数，该怎么数。

结论（先看）

1. 该芯片的 PS 侧没有 AFI 模块，也不提供 `S_AXI_HP*_RCOUNT` / `RACOUNT`；AXI 协议同样不定义"在途事务计数"信号。
2. 所以"PS 侧还剩多少在途事务"读不到，只能在 PL 侧自行计数。
3. 自己写计数器可行；更省事的做法是给 PL master 加忙/闲状态输出（判空握手，见 2.5 问 3）。

1.1 计数器怎么做（已在 System ILA 中验证的一套）

本节给出一套可直接照搬的计数定义，供自行实现时对照。

1.1.1 计数规格

项目	定义
输出路数	对外 4 路：读 `arcnt` / `rcnt`，写 `awcnt` / `bcnt`
计数类型	up-down 的"在途计数"，不是累计计数；接口空闲时必须为 0
计数粒度	事务级（transaction），不是 beat 级
加 1 条件	读 `ARVALID && ARREADY`；写 `AWVALID && AWREADY`
减 1 条件	读 `RVALID && RREADY && RLAST`；写 `BVALID && BREADY`
完成判定	与 AMBA 事务完成判定完全一致
溢出行为	回绕（wrap），且没有任何回绕 / 溢出标志位

表下两点说明：

- 这 4 路不是 AXI 标准信号，不受协议约束。芯片不提供，需要在 PL 侧取得（例化 `gigantic_mux`，或使用 System ILA 自带的 "AXI4 Transaction Tracking"）。
- 因为没有溢出标志位，不能靠"计数变小了"判断它是否已回绕。

1.1.2 精确公式

```
读在途 = (ARVALID && ARREADY) - (RVALID && RREADY && RLAST)
写在途 = (AWVALID && AWREADY) - (BVALID && BREADY)
```

即：每收到一次请求加 1；读通道收到"最后一次数据"（`RLAST`）减 1，写通道收到写响应（`B`）减 1。

两个例外：

- 只有 `AXI4LITE` 槽位的读减计数是 `RVALID && RREADY`（不看 `RLAST`）；
- `AXI_ID_WIDTH > 0` 时，读减计数还要 `RID` 命中、写减计数还要 `BID` 命中（按 ID 配对），原因见 1.1.3。

1.1.3 四路输出是什么关系

情况	四路的关系	能否代表在途总数
`AXI_ID_WIDTH == 0`	读两路恒等，写两路恒等	能，任取一路即可
`AXI_ID_WIDTH > 0`	四路两两不相等	不能，每路只是"最后被碰到的那条线程"的计数
自己写的计数器	不按 ID 分线程，只有一份计数	能，就是总数

为什么 `ID > 0` 时四路都不等于总数：这套计数器是按 ID 分线程实现的：每个 ID 占一个线程槽、各数各的，线程数 = `min(2^ID_WIDTH, 计数位宽)`；一次加或减只动它自己那条线程，四路输出各自只是"最后被碰到的那条线程"的读数。两次碰到的若不是同一条线程，两路读数就不同，也都不等于各线程之和。

依据：`AXI_ID_WIDTH > 0` 时减计数要求 `RID` / `BID` 命中（同一 ID 才配对），这正是"线程槽"实现的体现。

1.1.4 落地建议

- `ID == 0`：任取一路即真实在途数，可直接用来判"还剩多少 / 是否排空"。
- `ID > 0`：三条路可走 —— ① 在 PL 侧按 ID 求和；② 自己写计数器时不按 ID 分线程（最直接）；③ 改用判空握手（见 2.5 问 3）。
- 计数器的复位源不要接到无关复位上，否则"被清零"会伪装成"已排空"。

1.2 客户问题与答复（计数）

客户问题（原文）

> `S_AXI_HP*_RCOUNT` / `RACOUNT` 分别统计什么？是否包括已接收但尚未完成的 AXI 读事务？计数归零能否保证 PS/AFI 内部已经没有未完成的读事务、或待返回的 `RDATA`/`RVALID`？

答复（按客户问题的三个分句逐条回答）

（1）这两路分别统计什么？

无定义。该芯片的 PS 无 AFI 模块，也不提供这两个信号。下面（2）（3）说的是"在 PL 侧自行实现计数"时的情况。

（2）是否包括已接收但尚未完成的读事务？

包括，全部计入。这正是"在途（outstanding）"的定义：`AR` 已握手、尚未收到 `RLAST` 的读事务都算在内。该计数需要在 PL 侧自行实现。

（3）计数归零能否保证 PS 侧已无未完成读事务、不会再返回旧 `RDATA` / `RVALID`？

可以，前提是计数逻辑正确：

- 加 / 减条件与 1.1.2 一致；
- 计数器的复位源不接无关复位，避免"被清零"被当成"已排空"；
- `ID > 0` 时四路都不代表总数，需另行取总数（1.1.3）；自己写计数器时可不按 ID 分线程。

依据：检查期间不再发起新事务，计数只减不增，因此不会出现"因回绕而假归零"；剩下的风险只有两条 —— 计数语义写错、或复位源接错。

建议做法

给 PL master 加上"使能输入"和"总线忙 / 闲状态输出"（即判空握手，见 2.5 问 3）：

1. 关闭 master：停止发起新传输，已发出的让它自然完成；
2. 读总线状态；
3. 等它变为 idle；
4. 再执行复位。

---

2. 复位：会发生什么、该怎么办

这一节回答：复位有哪几种、各自会发生什么、每种该怎么操作。

2.1 三条协议层常识

1. AXI 不要求"复位时把已发出的在途事务做完"。复位后事务是完成还是被丢弃，取决于具体实现，master 不能假设它一定完成。
2. 任一侧在非空闲时被复位，另一侧会一直等下去：

谁被复位	另一侧的等待位置
PL 被复位	PS 侧 slave 一直等写数据 `WVALID`，或等读数据被接收
PS 被复位	PL 侧 master 一直等写被受理、等写响应 `B`、等读数据与 `RLAST`

3. 因此只有两种安全做法：在接口空闲时复位；或者把 PS 与 PL 一起复位。

2.2 复位与隔离的四条事实

适用范围：下面四条事实，以及全文提到的"设计1 / 设计2 / 设计3"，都以 **PL 复位由 `processing system reset`（`proc_sys_reset`）IP 产生**为前提。如果复位不是用这个 IP（例如自建复位同步、复位直接来自管脚或其它逻辑），"PS 复位会不会连带复位 PL"需要按同样的思路具体分析，不能直接套用下面的分档。

事实 1：PS 复位会不会连带复位 PL，取决于设计

这里的"设计"指的是 PL 的时钟与复位从哪里来：

设计	PL 的时钟 / 复位来自	PS 复位时 PL
设计1	都来自 PS（都要经 `USER_LEVEL_SHIFT`）	不跟随复位
设计2	复位来自 PS，时钟来自管脚	跟随复位
设计3	都与 PS 无关	不跟随复位

设计2 之所以跟随复位，是因为它的复位来自 PS：PS 复位会关掉 `USER_LEVEL_SHIFT`，并把 `FCLK_RESETN` 钳在低电平，PL 因此被一起复位。

复位 PL 有两条途径，都不影响 PS：

- 逻辑复位：写 `FPGA_RST_CTRL`。只清 PL 逻辑，配置保留，恢复快。
- 重配置：拉 `PROGRAM_B`。逻辑与配置一起重建，覆盖彻底。

运行期没有"一个动作同时清掉两侧"的开关（设计2 是例外，它的 PS 复位本身就带走了 PL）。要让两侧都清：

- 设计1：按 2.4 的 L1 流程用软件复位 PL 即可。分两次动作，效果与 L5 相同。
- 设计3：PS 侧无法复位 PL，只能整机上下电，或人为成对触发。

事实 2：HP 端口没有独立的复位引脚

HP 端口的复位由 PS 侧寄存器 `AXI_HP_RST_CTRL` 控制，复位对象是 HP 端口侧的 NIC 桥（客户所称 "AFI"），时机由软件决定。这就是后文的 L2。

事实 3：PS 复位会把 `USER_LEVEL_SHIFT` 清零

这一清零把 PS-PL 链路切断，PS→PL 与 PL→PS 的信号都被钳位；要等 PS 内部配置完成、软件重新打开开关，链路才恢复。

这个开关是全局的：它用 4 位寄存器管住全部 PS-PL 信号，不是每组 HP 端口一位。所以一关就是 GP、ACP、中断、DMA 一起断。

关键后果（设计1 前提，即 PL 的时钟与复位都取自 PS）：PL 的时钟与复位也来自 PS，也走这个开关。因此隔离期间 PL 既没有时钟、也没有复位，完全不工作。

依据：PL 时钟 `FCLK` 与复位 `FCLK_RESETN` 都由 PS 提供并经过该开关；`FCLK_RESETN` 进入 PL 后还要过 `processing system reset`（`proc_sys_reset`）的 4 级滤波与同步（默认 4 拍）才生效，也就是说复位生效本身也要靠 PL 有时钟。开关一关，时钟与复位同时断，PL 内所有状态机自然都不动。

事实 4：`AXI_HP_RST_CTRL` 清不掉下游残留

它的复位对象只有 HP 端口侧的 NIC 桥：不复位 PS 其他部分、不复位 PL、不断链路，也不涉及 DDR 控制器及其他 AXI 组件。

如果在途事务已经越过 NIC 桥、进入下游（例如 DDR 控制器），写这个寄存器清不掉它。该处会一直持有未完成传输，可能导致异常，甚至把系统总线挂死（会波及 CPU 与其他主设备）。

所以寄存器复位不能替代排空，只能在排空之后使用。

2.3 五种场景对比

场景	复位 PS	复位 PL	PS-PL 链路
L1 PS 系统复位	是	与设计相关	被切断，PS→PL 与 PL→PS 的信号都被钳位
L2 写 `AXI_HP_RST_CTRL`	否	否	不受影响，保持连通
L3 PL 重配置	否	是	保持连通
L4 PL 逻辑复位	否	是	保持连通
L5 整机上下电 / 成对复位	是	是	复位期间同样被切断

三条读表要点：

- L1 下 PL 挂死的直接原因是"链路被切断"，而不是"PS 拒绝响应"。
- L3 与 L4 都会让 PS 侧在途事务无人消费，形成背压，并波及系统总线。
- 只有 L5（两侧一起清）没有残留，因此最安全。

2.4 五个场景怎么做

L1：PS 系统复位

【什么情况会遇到】 PS 因 POR、SRST 或看门狗等原因整体复位。PL 会不会跟着复位，取决于三种设计（回顾事实 1；该分档的前提是 PL 复位用 `proc_sys_reset`，其他接法需具体分析）：

设计	PL 的时钟 / 复位来自	L1 下 PL 会被复位吗	L1 之后要做什么
设计1	都来自 PS	不会	按下面的 8 步用软件复位 PL
设计2	复位来自 PS、时钟来自管脚	会（被连带复位）	不需要额外动作
设计3	都与 PS 无关	不会，且 PS 侧无法复位它	只能整机上下电或人工成对触发（见 L5）

【会发生什么】

1. 链路被切断，PS→PL 与 PL→PS 的信号都被钳位，PL 侧看到的 `ARREADY` / `AWREADY` / `RVALID` / `BVALID` 全低。
2. PS 侧的 NIC 桥随 PS 一起复位，PS 侧不残留旧事务状态。
3. 设计1 / 设计3 下 PL 没有被复位：它既发不出新请求，也收不到任何响应，不处理就是挂死。

【怎么处理】 设计1 下，由软件在 PS 配置完成后复位 PL，顺序如下（设计2 无需执行；设计3 下 PS 侧做不到，见 L5）：

1. PS 复位。链路自动断开（`USER_LEVEL_SHIFT` 清零），PS→PL 与 PL→PS 的信号都被钳位。
2. 等 PS 内部配置完成。这期间 PL 不得发起任何 AXI 访问。
3. 写 `AXI_HP_RST_CTRL = 0xF`，断言 HP / NIC 桥复位。必须在第 5 步之前。
4. 写 `FPGA_RST_CTRL = 0xF`，断言 PL 复位。必须在第 5 步之前。
5. 写 `USER_LEVEL_SHIFT = 0xF` 连通。PL 同时拿到时钟和低电平复位，它的状态机、指针、FIFO 在这一步被清零。
6. 写 `AXI_HP_RST_CTRL = 0x0`，释放 HP / NIC 桥复位。
7. 确认 PL 已复位（或等足够时间让复位完成），写 `FPGA_RST_CTRL = 0x0` 释放 PL 复位。
8. PL 侧开始重新发起事务。

【注意】

- 第 3、4 步必须排在第 5 步之前：先断言、再连通，复位信号才会在连通那一刻对 PL 有效。设计1 下链路断开时断言送不进 PL。
- 复位要保持足够多拍才生效：`proc_sys_reset` 的复位输入要过 4 级滤波与同步（默认 4 拍），而这些拍数只能在 PL 有时钟时累积，所以复位要在连通前置起、连通后保持多拍。
- 设计3 下 PS 侧无法复位 PL，只能靠读超时脱困、由 PL 状态机自复位，不建议采用这种设计。

L2：只写 `AXI_HP_RST_CTRL`（HP 接口复位）

【什么情况会遇到】 软件主动复位 HP 端口侧的 NIC 桥，用来清掉滞留在桥内的事务。

【会发生什么】 按事务所在位置分两种：

事务位置	能否清掉	结果
还在 NIC 桥内	能	事务被丢弃。对 PL 就是不返回响应，PL 只能靠读超时脱困
已越过 NIC 桥进入下游	不能	该处继续持有未完成传输，可能导致异常乃至系统总线挂死

【怎么处理】 写 `AXI_HP_RST_CTRL = 0xF` 断言，再写 `0x0` 释放。

【注意】

- 不能替代排空：下游残留清不掉，所以不要在"还有在途传输"时把它当应急手段。
- 它只适合在排空之后使用。
- 它可以用来单独隔离 PL 对 HP 接口的影响。

L3：PL 重配置（`PROGRAM_B`）

【什么情况会遇到】 需要重新加载 PL 位流时。

【会发生什么】 重配置不清 `USER_LEVEL_SHIFT`，也就是既不复位 PS、也不断开链路。PS 侧的在途事务会一直留着，等一个已经消失的 PL 来消费 —— 背压因此成为确定结果，并波及系统总线。现场"重配置后 PS 侧总线不动"就是这个原因。

依据：`PROGRAM_B` 只重建 PL 的逻辑与配置，不影响 PS 侧状态机，也不改变链路连通性，所以 PS 侧没有任何机制去感知或清理这些在途事务。

【怎么处理】 准入条件是"重配置前已排空"，而且重配置前必须先关断链路：

1. 停止发起新的 HP 事务（PL 不再 push）。
2. 等在途事务自然完成，直到 master 自身"未完成事务"为 0 并保持。
3. 确认排空后，写 `USER_LEVEL_SHIFT = 0x0` 关断 PS-PL 连通，使 PS 不受 PL 配置过程干扰。
4. 执行 `PROGRAM_B` 重配置。
5. 重配置后复位 PL 事务状态机：写 `FPGA_RST_CTRL = 0xF` 断言，写 `USER_LEVEL_SHIFT = 0xF` 连通，确认后写 `0x0` 释放。
6. 确认 PL 状态机已重新初始化，再开始发起事务。

【注意】

- 第 3 步不能省：不关断就重配置，PS 侧的总线可能受 PL 配置过程中的信号扰动影响。
- 重配置没有自动屏障，唯一可靠的保护就是"先排空"。

L4：PL 逻辑复位（断言 PL 复位信号）

【什么情况会遇到】 只想清 PL 内部逻辑、不想重新加载位流时。

【会发生什么】

1. PS 侧同样没有复位；PL 复位后不再消费读数据，PS 侧照样背压，并波及系统总线。
2. 复位范围就是这个复位信号的扇出：没接该复位网的模块（含其内部 FIFO、指针）不会被清，可能残留旧状态。

【怎么处理】 准入条件同样是"复位前已排空"，但链路本来就连通，不需要关断：

1. 停止发起新的 HP 事务。
2. 等在途事务自然完成，直到 master 自身"未完成事务"为 0 并保持。
3. 确认排空后，写 `FPGA_RST_CTRL = 0xF` 断言 PL 复位，保持多拍后写 `0x0` 释放。
4. 确认 PL 状态机已重新初始化，再开始发起事务。

【注意】

- 与 L3 的差别只在恢复更快：配置还在，不用重加载位流，PLL 不丢锁。
- 未接该复位网的模块要单独处理：纳入复位网，或由软件显式清理。

L5：整机上下电，或 PS 与 PL 成对复位

【什么情况会遇到】 整机上下电 / POR，或者人为成对触发 PS 与 PL 复位。

【会发生什么】 两侧状态全清：PS 侧（含 NIC 桥及其下游）没有残留，PL 侧 master 状态机重建。既没有 PL 挂死，也没有背压，而且不需要"重配置前排空"。

【怎么处理】 与 L1 同一顺序，只在中间多一步 PL 重新配置：

1. PS 复位。链路自动断开，PS→PL 与 PL→PS 的信号都被钳位。
2. 等 PS 内部配置完成。这期间 PL 不得发起任何 AXI 访问。
3. 写 `AXI_HP_RST_CTRL = 0xF` 断言 HP / NIC 桥复位。必须在第 6 步之前。
4. 写 `FPGA_RST_CTRL = 0xF` 断言 PL 复位。必须在第 6 步之前。
5. 若重上电或位流已被清除，重新配置 PL（位流由 PS 加载）。
6. 写 `USER_LEVEL_SHIFT = 0xF` 连通。PL 获得时钟与低电平复位，状态机被清零。
7. 写 `AXI_HP_RST_CTRL = 0x0` 释放 HP / NIC 桥复位。
8. 确认 PL 已复位，写 `FPGA_RST_CTRL = 0x0` 释放 PL 复位。
9. PL 侧开始重新发起事务。

【注意】 这是五种场景里最安全的一种。

2.5 客户问题与答复（复位处理）

先更正一个前提

> 该芯片的 PS 不含 AFI 模块，因此"PS/AFI 会如何处理"这一提法不成立。PS 复位作用于整个 PS；与"AFI 复位"对应的对象是 HP 端口侧的 NIC 桥。

问 1：Outstanding Read 遇到 Reset 时的处理机制

客户问题（原文）

> 当 PL 已发起 AXI Read（AR 握手完成）、但 PS 尚未返回全部数据或 `RLAST` 时，如果发生复位，PS/AFI 会如何处理这些未完成的事务？尤其希望区分： - 仅 PL 侧 FIFO / 读控制逻辑复位，PS HP 接口保持运行； - PS 或 HP/AFI 相关模块发生复位； - 整板断电重新上电。

答复

一句话：结局取决于复位落在哪一层，判据是"PS 侧有没有被清"。

- PS 侧被清：事务被丢弃，之后不再返回。
- PS 侧没被清：PS 仍持有它，会继续把读数据推给 PL。
- 只清 HP 侧 NIC 桥：桥内的被丢弃；已进入下游其他 AXI 组件的残留清不掉。

按客户所列的三种情形逐条对照：

情形	事务结局	你要做什么
(a) 仅 PL 侧 FIFO / 读逻辑复位（L4；若整块 PL 重配置则为 L3）	不会自动丢弃：PS 仍会尝试把读数据推给 PL	停发、排空后再做复位 / 重配置，见 2.4 的 L3 / L4
(b)① PS 系统复位（L1）	被丢弃，之后不再返回	设计1 / 设计3：由软件随后复位 PL，否则 PL 会一直等 `RLAST` 而挂死；设计2 下 PL 已连带复位
(b)② 只写 `AXI_HP_RST_CTRL`（L2）	桥内的被丢弃；已进下游的清不掉	不能替代排空，只能在排空之后使用
(c) 整板断电重新上电（L5）	被丢弃，之后不再返回	无残留，不需要先排空

问 2：复位后是否可能出现旧读响应

客户问题（原文）

> 在 PL 侧恢复并重新发起读请求后，之前未完成事务对应的 `RVALID`/`RDATA`/`RLAST` 是否可能继续返回，或被新的读控制逻辑接收？芯片是否提供事务清空、响应隔离或复位后不保留旧响应的明确保证？

答复

（1）旧读响应是否可能继续返回、或被新的读逻辑接收？

可能。判据同样是"PS 侧有没有被清"：

- 只复位 PL（L3 / L4）：PS 侧没清，旧读响应可能在任意时刻返回。此时 PL 已在发新请求，新旧响应走同一条读数据通道，协议上无从区分，旧数据可能被新的读逻辑误收。
- PS 系统复位（L1）、整板上下电（L5）：不会。NIC 桥与下游一起被清。
- 只写 `AXI_HP_RST_CTRL`（L2）：不保证。桥内的会被清，但已进入下游其他 AXI 组件的读仍可能返回。

（2）芯片是否提供"事务清空 / 响应隔离 / 复位后不保留旧响应"的保证？

不提供。逐项说明：

- 计数侧没有任何回绕、溢出、清空或粘滞标志位（见 1.1.1）；
- `AXI_HP_RST_CTRL` 的复位对象只有 HP 端口侧的 NIC 桥，不涉及 DDR 控制器及其他 AXI 组件；
- 读通道没有错误响应，slave 被复位不会通知 master；
- PS 侧不暴露任何"读通道已清空"的状态：无 AFI 模块、无计数信号、无状态寄存器。

问 3：推荐的安全恢复流程

客户问题（原文）

> 如果系统需要在运行过程中恢复 PL 侧 AXI 读通道，企业是否有推荐的操作顺序（例如停止新请求、等待 outstanding read 完成、确认接口空闲、执行复位等）？是否存在能够可靠判断 PS/AFI 读通道已经清空的状态信号？哪些情况下必须执行更大范围的 PS 复位或整板重启？

答复（按三个分句）

（1）推荐的操作顺序

按 2.4 的流程执行：只做 PL 逻辑复位时用 L4 的四步；需要重配置时用 L3 的六步。两套流程前两步相同：先停发，再等在途事务归零。

（2）有没有"可靠判断 PS 读通道已清空"的状态信号?

没有。PS 侧不暴露任何此类状态：无 AFI 模块、无计数信号、无状态寄存器。

唯一可靠的来源是 PL master 自身维护的"未完成事务"状态（AXI 本来就要求 master 自己维护），把它引出来即可 —— 也就是判空握手：对外给出使能位、状态寄存器或状态管脚。

（3）哪些情况必须做更大范围的复位?

情况	必须的动作
复位 / 重配置前能由软件先排空	按 2.4 的流程执行即可，不需要复位 PS
无法确认排空，且复位 / 重配置由外部按键、JTAG 触发（软件来不及先排空）	必须成对复位 PS 与 PL，或在 PS 复位后由软件复位 PL

---

3. 相关手册

手册	用途
《UG601-JFMQL智能多媒体芯片_应用指南》	芯片与接口的应用说明
《UG602-JFMQL智能多媒体芯片_寄存器指南》	寄存器定义，包括 `AXI_HP_RST_CTRL`、`FPGA_RST_CTRL`、`USER_LEVEL_SHIFT`